

AI Agent 智能体系统 · 项目分工与技术文档

课程项目：人工智能设计 · Agent 设计
团队规模：5 人
项目周期：冬季学期

一、项目概述

本项目实现了一个能调用工具干活的 AI 智能体框架：用户的提问由远端大模型（Qwen）负责思考与决策，本地只负责执行工具（计算、联网、读写文件、代码运行等）。

核心特性

- **ReAct 闭环**：思考 → 调工具 → 观察结果 → 继续思考 → 最终回答
- **零本地算力**：所有智能来自云端 LLM，本地只执行工具
- **零第三方依赖**：仅用 Python 3.9+ 标准库，拿到就能跑
- **18+ 内置工具**：计算、搜索、文件、代码、知识库、记忆、多模态等
- **现代化 Web 界面**：三栏布局、流式输出、工具调用可视化

二、技术栈

后端技术

技术	说明
Python 3.9+	主开发语言，零第三方依赖
<code>urllib</code>	HTTP 客户端，调用 LLM API、网页抓取
<code>http.server</code>	标准库 HTTP 服务，SSE 流式接口

sqlite3	长期记忆持久化（标准库自带）
ast	计算器安全求值（AST 白名单防注入）
subprocess	代码执行（run_python / shell）

大模型接口

技术	说明
Qwen（阿里云百炼）	默认 LLM，OpenAI 兼容接口
Function Calling	函数调用，让 LLM 决定何时用哪个工具
SSE 流式输出	Server-Sent Events，打字机效果
text-embedding-v3	向量嵌入，用于知识库 RAG
Qwen-VL	多模态图像理解

前端技术

技术	说明
HTML5 / CSS3	单文件前端，无框架
原生 JavaScript	无 jQuery / 无 React
EventSource API	浏览器端 SSE 流式接收
响应式设计	桌面 + 手机自适应

存储与数据

技术	说明
SQLite	长期记忆（用户偏好 + 事实存储）
JSON 文件	知识库向量数据、用量统计历史

文件沙箱	workspace/ 目录隔离，防路径穿越
------	-----------------------

三、核心知识点

3.1 Agent 架构基础

知识点	说明	对应模块
ReAct 模式	Reasoning + Acting 交替进行	agent/core.py
感知-决策-执行	三层架构：输入 → 思考 → 行动	agent/core.py
函数调用 (Function Calling)	LLM 输出工具名 + 参数，本地执行	llm/base.py
工具注册表模式	统一管理工具的注册、列举、执行	tools/registry.py

3.2 记忆系统

知识点	说明	对应模块
短期记忆	对话上下文，token 滑动窗口裁剪	agent/memory.py
长期记忆	跨会话持久化，用户偏好 + 事实	agent/long_term_memory.py
自动记忆提取	LLM 自动从对话中提取值得记住的信息	agent/memory_extractor.py
用户画像	偏好注入 system prompt，个性化回答	agent/core.py

3.3 知识库 RAG

知识点	说明	对应模块
RAG (检索增强生成)	先检索相关文档，再拼入 Prompt	agent/rag.py
文档切分	按段落/句子切分，带重叠	agent/rag.py

向量嵌入 (Embedding)	文本转成稠密向量	llm/embedding.py
余弦相似度检索	计算向量相似度，取 top-k	agent/rag.py

3.4 智能增强

知识点	说明	对应模块
任务规划	复杂任务拆解成步骤计划	agent/planner.py
自我反思	工具失败后回传 LLM 自动调整	agent/core.py
自动重试	网络错误类工具自动重试	agent/core.py
遥测与成本统计	token 计数、费用估算、运行日志	agent/telemetry.py

3.5 高级能力

知识点	说明	对应模块
多Agent协作	主管-员工模式，任务分配与汇总	agent/multi_agent.py
工作流编排	多步骤批量执行，变量引用	tools/workflow.py
多模态	图像理解 (Qwen-VL)、文生图	tools/multimodal.py
代码执行	Python / Shell 执行 + 授权机制	tools/executor.py

3.6 安全设计

知识点	说明	对应模块
AST 白名单	计算器只允许数学运算，防代码注入	tools/builtin.py
路径沙箱	文件工具只能访问 workspace/	tools/builtin.py
授权机制	代码执行需用户确认 (ask/auto/off)	tools/executor.py
风险黑名单	删除、格式化、关机等高危操作拦截	tools/executor.py

四、五人分工

成员 A：核心引擎组

负责模块：Agent 核心循环、LLM 接口、短期记忆

文件	职责
agent/core.py	ReAct 主循环、工具调用调度、最终回答
agent/memory.py	上下文滑动窗口、token 估算、消息裁剪
llm/base.py	LLMClient 抽象接口、TokenUsage 数据类
llm/qwen_client.py	Qwen API 调用、SSE 流式、usage 解析
llm/mock_client.py	离线模拟客户端
config.py	配置加载、环境变量解析

核心知识点：ReAct 模式原理与实现、函数调用协议、SSE 流式输出、token 预算管理

交付目标：Agent 能正常对话、调用工具、流式输出；支持切换不同 LLM 后端

成员 B：工具系统组

负责模块：基础工具、代码执行、安全沙箱

文件	职责
tools/base.py	Tool 抽象基类
tools/registry.py	工具注册表（注册/列举/执行）
tools/builtin.py	计算器、时间、网页抓取、网页搜索、文件读写

`tools/executor.py`

run_python / shell 代码执行 + 授权器

核心知识点：插件化工具系统、AST 安全求值、路径沙箱、代码执行授权策略

交付目标：9+ 个内置工具全部可用；计算器安全；文件操作不能越界；代码执行有授权确认

 成员 C：智能增强组

负责模块：知识库 RAG、长期记忆、任务规划、遥测统计

文件	职责
<code>agent/rag.py</code>	文档切分、向量存储、余弦相似度检索
<code>llm/embedding.py</code>	Qwen Embedding API 客户端
<code>agent/long_term_memory.py</code>	SQLite 持久化、用户偏好、事实记忆
<code>agent/memory_extractor.py</code>	自动从对话提取记忆
<code>agent/planner.py</code>	任务规划器、启发式判断
<code>agent/telemetry.py</code>	token 统计、成本估算、运行日志
<code>tools/knowledge.py</code>	knowledge_search / add / list 工具
<code>tools/long_term_memory.py</code>	remember / recall / forget 工具

核心知识点：RAG 完整流程、向量嵌入、长期记忆与用户画像、任务规划、遥测

交付目标：知识库能添加文档、语义检索；长期记忆跨会话生效；复杂任务自动拆解

 成员 D：高级能力组

负责模块：多Agent协作、工作流编排、多模态工具

文件	职责
agent/multi_agent.py	主管-员工模式、子Agent调度
tools/workflow.py	工作流执行器、变量引用、条件分支
tools/multimodal.py	图像理解（Qwen-VL）、文生图

核心知识点：多Agent协作架构、工作流编排、多模态大模型、文生图 API

交付目标：多Agent模式能拆解复杂任务；工作流工具能一次执行多步骤；图像理解 + 文生图

 成员 E：前端与部署组

负责模块：Web 前端、HTTP 服务、测试、文档

文件	职责
web/agent-chat.html	现代化三栏布局聊天界面
server.py	HTTP 服务、SSE 接口、API 路由
main.py	命令行入口
factory.py	装配工厂（所有模块组装）
tests/test_all.py	全模块测试用例
README.md	项目文档

核心知识点：原生 JS 单页应用、EventSource API、HTTP 服务设计、响应式 CSS、软件测试

交付目标：Web 界面美观好用；所有 API 接口文档齐全；测试用例覆盖率 90%+

五、协作流程

开发顺序

第一周：核心引擎 + 基础工具（A + B）
↓
第二周：智能增强（C）
↓
第三周：高级能力（D）
↓
第四周：前端 + 集成测试（E + 全员）

Git 协作

- 每人一个 feature 分支，开发完合并到 main
- 提交信息规范： **feat**: 新增功能 / **fix**: 修复bug / **docs**: 文档更新
- 每天至少 push 一次，避免代码丢失

接口约定

- 模块之间通过抽象接口解耦（如 LLMClient、Tool、Agent）
- 新增工具继承 Tool 基类，注册到 ToolRegistry 即可
- 新增 LLM 后端继承 LLMClient 即可切换

六、项目结构总览

AgentDesign/
├─ agent/ # Agent 核心逻辑
│ ├── core.py # ReAct 主循环
│ ├── memory.py # 短期记忆（滑动窗口）
│ ├── long_term_memory.py # 长期记忆（SQLite）
│ ├── memory_extractor.py # 自动记忆提取
│ ├── rag.py # 知识库 RAG
│ ├── planner.py # 任务规划器
│ ├── multi_agent.py # 多Agent协作
│ └─ telemetry.py # 遥测与统计
├─ llm/ # 大模型接口
│ └─ base.py # 抽象接口

```
|   ├── qwen_client.py      # Qwen 实现
|   ├── embedding.py       # 向量嵌入
|   └── mock_client.py     # 离线模拟
├── tools/                 # 工具系统
|   ├── base.py           # 工具基类
|   ├── registry.py       # 注册表
|   ├── builtin.py        # 基础工具
|   ├── executor.py       # 代码执行
|   ├── knowledge.py      # 知识库工具
|   ├── long_term_memory.py # 记忆工具
|   ├── workflow.py       # 工作流工具
|   └── multimodal.py     # 多模态工具
├── web/                  # 前端
|   └── agent-chat.html   # 单文件聊天界面
├── tests/                # 测试
├── config.py             # 配置
├── factory.py            # 装配工厂
├── main.py               # 命令行入口
├── server.py             # HTTP 服务
└── README.md             # 项目文档
```

七、评分要点（建议）

维度	占比	说明
功能完整性	30%	所有模块是否正常工作
代码质量	20%	结构清晰、注释充分、零依赖设计
技术深度	20%	RAG、多Agent、ReAct 等原理的理解
前端体验	15%	界面美观、流式输出、工具可视化
文档与测试	15%	README、测试用例、分工文档

文档生成时间：2026-10-08

项目仓库：<https://github.com/CHD-Avril/AgentDesign>